

GMC AQI / WEATHER STATION

Firmware Handover & Technical Documentation

Document Title	GMC AQI / Weather Station – Firmware Handover & Technical Documentation
Author	Yash Yadav
Department	Embedded
Date of Issue	07/09/2026
Version	1.0
Document No.	ENG-AQI-0001
Document Classification	Internal
Document Status	Ongoing

Project: GMC_AQI_UPDATED_V1

Target Hardware: ESP32-S3-DevKitC-1

Preface

This document is an official technical record of Immensystech Private Limited. It provides the firmware handover, architecture, hardware interfaces, sensor drivers, communication paths, scheduling behavior, deployment procedure, troubleshooting guidance and implementation findings for the GMC AQI / Weather Station project.

The document is intended for firmware engineers, hardware engineers, validation teams and project stakeholders responsible for maintaining, testing or deploying the AQI system.

Confidentiality & Legal Notice

This document may contain proprietary engineering information, internal methodologies, firmware architecture, hardware interface details and network configuration information. Unauthorized access, disclosure, copying, distribution or external use is prohibited without appropriate authorization.

Version Control & Document Governance

This document is maintained under a controlled version system. All modifications shall be recorded in the Revision History section. Users must verify that they are using the latest approved version before deployment or production use.

Revision History

Name	Description of Changes	Date	Version	Review By
Embedded Team	Initial AQI firmware technical documentation	07/09/2026	1.0	_____

Review By

Name	Position	Signature	Date
_____	_____	_____	_____

Approve By

Name	Position	Signature	Date
_____	_____	_____	_____

Table Of Contents

1. Purpose, Scope & Introduction
2. Executive Summary
3. Source Baseline
4. System Architecture & Data Flow
5. Hardware & Interface Map
6. Sensor Data Model
7. I²C HAL & Bus Allocation
8. Sensor Driver Details
9. Scheduler & Runtime Sequence
10. GSM & Remote Transmission
11. Ethernet, Wi-Fi & Web Server
12. OTA & Watchdog
13. Critical Handover Findings
14. Build & Flash Procedure
15. First-Power-On Validation Checklist
16. Troubleshooting
17. Recommended Next Actions
18. File-by-File Guide

1. Purpose / Scope / Introduction

The purpose of this document is to provide a complete technical handover of the supplied GMC AQI firmware. The firmware implements an ESP32-S3 based Air Quality Index (AQI) and Weather Station platform integrating gas sensors, particulate matter sensing, CO₂ measurement, environmental sensing, UV, ambient light, noise and RTC functions.

The scope includes firmware architecture, hardware interfaces, sensor drivers, scheduling, data handling, GSM/Ethernet/Wi-Fi communication, OTA support, watchdog handling, build and flash procedures, validation, troubleshooting and known implementation risks.

2. Executive Summary

The supplied firmware is a PlatformIO/Arduino project for an ESP32-S3-DevKitC-1 air-quality/weather station. It integrates particulate matter, gas, CO₂, temperature, humidity, pressure, UV, ambient light, noise and RTC functions.

Connectivity includes GSM TCP transmission, W5500 Ethernet support, a Wi-Fi HTTP configuration/live-data server, NVS configuration storage, OTA support and a task watchdog.

3. Source Baseline

- platformio.ini – build environment and dependencies.
- src/main.cpp – startup, scheduler, networking, web server and active GSM path.
- src/data_structure.h/.c – common sensor data structure.
- src/i2c_20k.h/.cpp – dual I²C HAL.
- src/sds011.* – SDS011 driver and recovery.
- src/tb600b.* – TB600B gas driver.
- src/s300e.* – S300E CO₂ driver.
- src/bme680.* and bme68x.* – BME680 wrapper and driver.
- src/ltr390.* – UV driver.
- src/veml7700.* – ambient-light driver.
- src/rtc_s.* – DS3231 driver.
- src/gas_sensor.* – gas/noise multiplexer path.
- src/gsm.* – alternate GSM implementation.
- src/ota.* – OTA support.
- src/timer.* – periodic scheduler flags.

4. System Architecture & Data Flow

Sensors → Driver Layer → Global weather_data Structure → Periodic Scheduler → JSON Builder / Transmission → Remote Server

Wi-Fi provides local HTTP configuration and live-data access. ESP32 Preferences / NVS persist network, server and data-interval configuration.

5. Hardware / Interface Map

Function	Interface	Pins / Address	Notes
I2C0	I ² C	SDA 47 / SCL 21	20 kHz; TB600B
I2C1	I ² C	SDA 18 / SCL 48	20 kHz; environmental sensors
SDS011	UART1	RX17 / TX16	9600 baud
Gas/Noise MUX	UART	RX38 / TX39	S0=40, S1=41, EN1=36, EN2=37
GSM Active Path	HardwareSerial(0)	RX43 / TX44	115200 baud
W5500	SPI	CS10 / RST7	Ethernet

6. Sensor Data Model

All sensor drivers write measurement results into the global `weather_data` object based on `weather_station_global_structure_t`. This shared structure acts as the common measurement container used by acquisition, JSON generation and communication functions.

7. I²C HAL & Bus Allocation

The firmware initializes two I²C buses as masters using internal pull-ups and a 20,000 Hz clock. Register and raw transactions use a 100 ms command timeout.

- I2C0 APIs are used for TB600B gas sensors at addresses 0x20–0x23.
- I2C1 APIs are used for BME680 0x76, LTR390 0x53, VEML7700 0x10, DS3231 0x68 and S300E 0x31.

Hardware validation is required to confirm PCB pull-ups, voltage levels and the 20 kHz bus setting.

8. Sensor Driver Details

8.1 SDS011 PM

Uses UART1 at 9600 baud. The driver validates 10-byte frames, frame type 0xC0 and checksum. PM2.5 and PM10 raw values are divided by 10. Startup includes a 30-second warm-up and recovery if valid packets are not received.

8.2 TB600B Gas Sensors

Four TB600B devices are connected on I2C0. Register 0xD2 provides decimal scaling, sensor type, unit and range. Register 0xD1 provides concentration, temperature and humidity with checksum validation. Concentration is calculated as raw value divided by 10^{decimal}.

8.3 S300E CO₂

Located at address 0x31. The driver writes ASCII 'R', waits 2 ms, reads 16 bytes, validates configuration byte 0x08 and reports CO₂ as 400 + `co2_delta`. The current implementation contains a 5-second blocking delay.

8.4 BME680

Located at 0x76. Configuration uses humidity 2x, temperature 4x, pressure 4x, filter 7, ODR none and gas heater disabled. A forced measurement is triggered followed by a 1-second delay.

8.5 LTR390 UV

Located at 0x53. Part ID 0xB2 is checked and the device is configured for UV mode. UV sensitivity is 2300 counts/UVI and UVI-to-irradiance conversion is 2.5 $\mu\text{W}/\text{cm}^2$ per UVI. Scheduled acquisition is currently commented out.

8.6 VEML7700 Light

Located at 0x10. Lux is calculated as raw ALS $\times$ 0.0576. Scheduled task execution is currently commented out.

8.7 DS3231 RTC

Located at 0x68. Seven BCD registers are read and converted to decimal before values are copied into `weather_data`.

8.8 Noise / Multiplexer Path

Uses UART RX38/TX39 and multiplexer controls S0=40, S1=41, EN1=36 and EN2=37. The periodic task selects the noise sensor and reads `NS_db`.

9. Scheduler & Runtime Sequence

The hardware timer generates periodic flags and `main.cpp` services those flags in the main loop. The implementation is flag-based scheduling rather than an RTOS task scheduler.

- 5 seconds: RTC read, firmware version print; UV/light calls exist but are commented out.
- 10 seconds: BME680 read, Wi-Fi reconnection and client cleanup.
- 30 seconds: Noise/gas acquisition, TB600B read, S300E read, SDS011 print and upload counter update.
- DATA_INTERVAL: When the 30-second counter reaches `DATA_INTERVAL  $\times$  2`, `gsmSendJSON()` is called.
- 40 seconds: GSM JSON build and print.

The intended transmission interval is approximately `DATA_INTERVAL` minutes.

10. GSM & Remote Transmission

The active implementation in `main.cpp` uses `HardwareSerial(0)`, RX43/TX44, APN `airtelgprs.com` and a configured TCP server. The sequence checks AT communication, SIM readiness, network registration, data activation, TCP connection, JSON transmission using `QISEND` and socket closure.

The JSON includes device ID, gas measurements, CO₂, PM2.5, PM10, lux, UVI, temperature, humidity, pressure and RTC date/time.

Security note: network credentials and configuration are present in plaintext in the supplied source and should be handled as confidential.

11. Ethernet, Wi-Fi & Web Server

W5500 uses CS10 and reset GPIO7. Network parameters and server settings are loaded from NVS before Ethernet initialization.

Wi-Fi provides an HTTP server on port 80 with routes `/`, `/set`, `/restart`, `/ota` and `/livedata`. NVS stores Ethernet, server, GSM and `DATA_INTERVAL` settings.

12. OTA & Watchdog

When `OTA_ENABLE_FLAG` is set, `ArduinoOTA.begin()` opens a 90-second OTA window. The task watchdog is configured for 30 seconds and reset repeatedly during OTA and normal operation.

Long blocking operations must be handled carefully to prevent watchdog resets. The current `S300E` and `BME680` functions contain blocking delays.

13. Critical Handover Findings

- Duplicate GSM implementations exist with different UART pins and constants. Reconcile before future changes.
- Device ID mismatch exists between `AQI_011` and `AQI_001`. Confirm the production device ID
- `LTR390` and `VEML7700` scheduled reads are commented out, allowing stale or zero values.
- `SDS011` supplies `PM2.5/PM10` only while other PM fields are zeroed.
- The I²C HAL deliberately uses 20 kHz and should be validated against all hardware.
- Wi-Fi credentials are hard-coded in `main.cpp`.

14. Build & Flash Procedure

- Open `GMC_AQI_UPDATED_V1` in VS Code with PlatformIO.
- Confirm environment [`env:esp32-s3-devkitc-1`].
- Connect the ESP32-S3 using USB and identify the serial port.
- Perform a clean build before release flashing.
- Flash firmware through PlatformIO.
- Open serial monitor at 115200 baud and verify initialization logs.
- Verify both I²C buses, sensors, `SDS011` warm-up, RTC, Wi-Fi, Ethernet and GSM registration.
- Verify TCP server connection and JSON reception.
- For OTA, configure `upload_protocol=esptota` and `upload_port` for the actual device/network.

15. First-Power-On Validation Checklist

- ESP32 boots successfully at 115200 baud.
- I2C0 and I2C1 initialize successfully.
- `BME680` provides valid temperature, humidity and pressure.
- `TB600B` CO/O3/NO2/SO2 connection checks pass.
- `LTR390` reports part ID 0xB2.
- `SDS011` produces valid PM frames after warm-up.
- `DS3231` returns valid date/time.
- Wi-Fi and HTTP server operate correctly.
- `W5500` reports the expected local IP.
- GSM SIM/network registration succeeds.
- TCP connection and JSON upload succeed.
- Sensor units and ranges are confirmed with the backend team.
- No watchdog resets occur during normal operation.

16. Troubleshooting

Symptom	Likely Cause	Recommended Action
I ² C sensor missing	Bus/address/wiring/pull-up/speed	Check bus/address map, SDA/SCL and 20 kHz setting.
SDS011 no data	UART wiring/power/invalid frame	Check RX/TX, power, warm-up and checksum logs.
TB600B missing	I2C0/address/checksum	Verify 0x20–0x23 and SDA47/SCL21.
CO ₂ read error	S300E transaction/frame issue	Verify 0x31, 'R' command, 16-byte frame and config 0x08.
GSM upload fails	SIM/network/APN/server	Check CPIN, registration, signal, APN, IP/port and TCP response.
Web server unavailable	Wi-Fi/server not active	Check Wi-Fi status, local IP and startup logs.
Watchdog reset	Blocking operation/timeouts	Inspect delays and AT waits; use non-blocking state machines.
Wrong device ID	AQI_011 vs AQI_001	Set one authoritative device ID.
UV/lux stale	Scheduled reads disabled	Re-enable and validate reads if required.

17. Recommended Next Actions

- Select one GSM implementation and retire the duplicate.
- Centralize device ID, APN, server endpoints and credentials.
- Move production credentials and network settings out of source where practical.
- Replace long blocking delays with timestamp/state-machine based tasks.
- Add sensor validity/status flags to the common data model and JSON.
- Correct or rename *_mg fields so units cannot be misinterpreted.
- Document the actual AQI calculation separately; no AQI-index calculation is visible in the supplied firmware.
- Re-enable and validate LTR390/VEML7700 if required.
- Add actual PCB pinout/revision drawing to the release package.
- Create a clean production release/tag and exclude generated .pio artifacts.

18. File-by-File Guide

File	Responsibility
main.cpp	Startup, runtime loop, scheduling, networking, web routes and active GSM path
data_structure.h/.c	Global sensor data definition and initialization
i2c_20k.h/.cpp	Dual I ² C initialization and transactions
sds011.h/.cpp	SDS011 protocol, parsing and recovery
tb600b.h/.cpp	TB600B gas sensors
s300e.h/.cpp	S300E CO ₂
bme680.h/.cpp + bme68x.*	BME680 wrapper and low-level driver
ltr390.h/.cpp	UV
veml7700.h/.cpp	Ambient light
rtc_s.h/.cpp	DS3231 RTC
gas_sensor.h/.cpp	Gas/noise multiplexer path
gsm.h/.cpp	Alternate GSM implementation
ota.h/.cpp	OTA support
timer.h/.cpp	Hardware timer and periodic flags
platformio.ini	Build configuration and dependencies

Conclusion

The supplied firmware is an integrated ESP32-S3 AQI/weather platform with dual-I²C abstraction and multiple communication paths. The highest-priority maintenance items are reconciling duplicate GSM implementations and configuration, confirming active sensor acquisition, and defining unambiguous units and status handling for production data.